KiteLang

A Basant-inspired language with a Flex/Bison interpreter
and an LLVM Ahead-of-Time compiler

Compiler Construction – Course Project
README: build, run and usage guide for Linux

Group member	Roll number
Mohammad Ibrahim Salman	23L-0723
Muzammil Ahmed Bhatti	23L-0688
Muhammad Ali Mahoon	23L-0531



Contents

1 Quick start (for the grader) 2

2 What KiteLang is 2

3 Installing dependencies (Linux) 3

4 Compiling the project 4

5 Running a KiteLang program 4

5.1 Phase 1 – Interpreter 4

5.2 Phase 2 – AOT compiler 5

5.3 Command-line usage 5

6 Using the language 5

6.1 Program structure and functions 5

6.2 Types, variables and arrays 6

6.3 Operators and output 6

6.4 Conditions and loops 6

6.5 `palta` – swap 7

7 Examples and testing 7

7.1 `examples/insertion_sort.kite` 7

7.2 `examples/array_smoke.kite` 8

7.3 Writing your own program 8

7.4 Automated test 8

8 Project files 9

9 Troubleshooting 10

10 Scope and known limitations 10

1 Quick start (for the grader)

If you only want to check that both phases work, run these commands from the project root on Ubuntu/Debian:

```
sudo apt update
sudo apt install -y build-essential flex bison clang llvm
make            # builds both phases into build/
make test       # runs Phase 1 and Phase 2 and compares output
```

What success looks like

`make test` prints no `diff` output and exits with status 0. That means the interpreter *and* the natively compiled program both printed `1 2 4 7 8 9` (one number per line), which matches `tests/expected_insertion_sort.txt`.

The rest of this guide explains every step in detail.

2 What KiteLang is

KiteLang is a small language for writing algorithm programs. It is inspired by *Basant*, the kite-flying festival. It keeps C-style functions, variables, arrays, expressions and braces, and replaces the control-flow keywords with kite-flying words:

Keyword	Meaning
`pecha`	`if`
`warna`	`else`
`dheel`	`while`
`dor`	inclusive, increasing range loop
`palta`	swap two assignable values

The project implements both required phases:

1. **Phase 1 – Interpreter.** Flex splits the source into tokens. Bison checks the grammar and builds an AST. C++ code inside the Bison grammar file then executes that AST directly. No intermediate code is produced.
2. **Phase 2 – AOT compiler.** The same parser turns the AST into textual LLVM IR (a `.ll` file). Clang compiles that IR into a native Linux executable.

Terms used in this guide

- **Lexer (Flex):** reads raw text and cuts it into *tokens*, the "words" of the language, such as `dor`, 42 or `+`.
- **Parser (Bison):** checks that the tokens follow the grammar rules and builds a tree from them.
- **AST (Abstract Syntax Tree):** a tree that represents the program's structure, e.g. "an `if` node with a condition child and a body child".
- **Interpreter:** a program that walks the AST and performs each action immediately.
- **LLVM IR:** LLVM's portable, assembly-like *intermediate representation*. Clang can turn it into machine code.
- **AOT (Ahead-of-Time) compiler:** translates the whole program into an executable *before* it runs.

Figure 1 shows how the two phases share the front end and split after the AST.

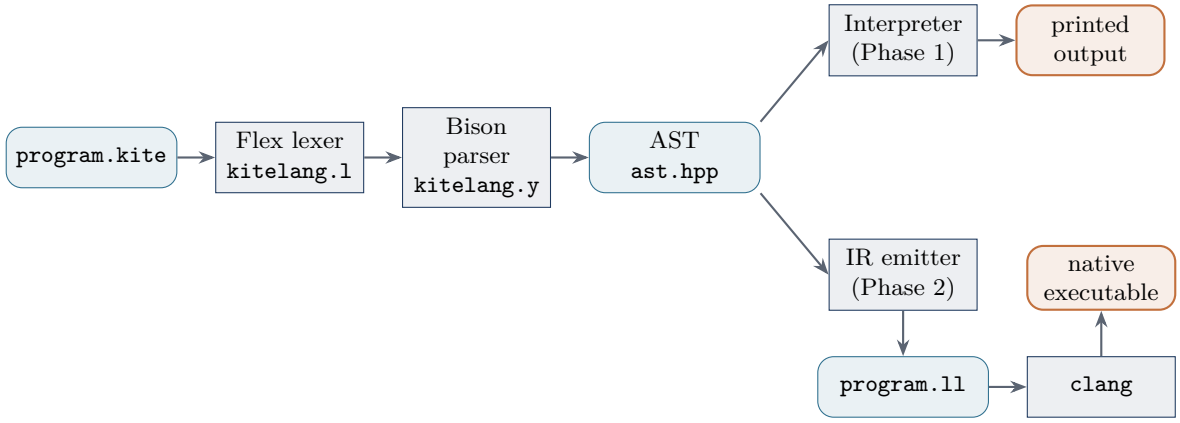


Figure 1: Both phases share the Flex lexer, Bison parser and AST. Phase 1 executes the AST directly; Phase 2 writes LLVM IR, which Clang turns into a native program. Rounded = data, square = processing step, orange = final output.

3 Installing dependencies (Linux)

You need `make`, a C++17 compiler (`g++`), Flex, Bison, Clang and LLVM. On Ubuntu/Debian:

```
sudo apt update
sudo apt install -y build-essential flex bison clang llvm
```

Check that the tools are installed:

```
flex --version
bison --version
g++ --version
clang --version
```

Clang version

The generated IR uses LLVM's *opaque pointer* type `ptr`. Use **Clang/LLVM 15 or newer**. Ubuntu 24.04 ships a new enough Clang by default. On an older release (for example Ubuntu 22.04, which defaults to Clang 14), install a newer version and use it in the Phase 2 commands:

```
sudo apt install -y clang-15
clang-15 build/insertion.ll -o build/insertion
```

`make test` calls plain `clang`, so on such a system either make `clang` point to the newer version or run the Phase 2 steps by hand (Section 5.2).

4 Compiling the project

From the project root:

```
make
```

This builds two programs from the same source files. The only difference is a preprocessor flag:

Executable	Phase	Built with flag
`build/kitelang-interpreter`	1 – interpreter	`-DMODE_INTERPRETER`
`build/kitelang-compiler`	2 – AOT compiler	`-DMODE_COMPILER`

What `make` does, step by step:

1. Creates the `build/` directory.
2. Runs `bison -d -o build/parser.tab.cpp src/kitelang.y`. This produces the parser and its header `build/parser.tab.hpp`.
3. Runs `flex -o build/lex.yy.cpp src/kitelang.l` to produce the lexer.
4. Compiles both generated files with `g++ -std=c++17 -Wall -Wextra -O2`, once for each mode.

All Makefile targets:

Command	Effect
`make`	build both phases
`make interpreter`	build Phase 1 only
`make compiler`	build Phase 2 only
`make test`	build and verify both phases end to end
`make clean`	delete the `build/` directory

The variables `FLEX`, `BISON`, `CXX` and `CXXFLAGS` can be overridden, e.g. `make CXX=clang++`.

5 Running a KiteLang program

5.1 Phase 1 – Interpreter

```
./build/kitelang-interpreter examples/insertion_sort.kite
```

Expected output:

```
1
2
4
7
8
9
```

The interpreter parses the file and runs `main` straight away. The value `main` returns becomes the process exit status (`return 0;` gives exit code 0). You can check it with `echo $?`.

5.2 Phase 2 – AOT compiler

```
./build/kitelang-compiler examples/insertion_sort.kite -o build/insertion.ll
clang build/insertion.ll -o build/insertion
./build/insertion
```

1. The first command parses the program, writes LLVM IR to `build/insertion.ll` and prints `generated build/insertion.ll`. If you leave out `-o`, the IR goes to `out.ll` in the current directory.
2. The second command uses Clang to compile the IR into a native executable.
3. The third command runs that executable. It prints the same six numbers as the interpreter.

Every generated `.ll` file starts with a format string and a declaration of C's `printf`, which `print` uses:

```
@fmt = private constant [4 x i8] c"%d\0A\00"
declare i32 @printf(ptr, ...)
```

Each KiteLang function then becomes one `define i32` (or `define void`) function. Integer parameters are `i32` and array parameters are `ptr`.

5.3 Command-line usage

```
kitelang-interpreter source.kite
kitelang-compiler   source.kite [-o output.ll]
```

If you run either tool with no arguments, it prints a usage message and exits with status 2. A file that cannot be opened also gives status 2. A syntax error gives status 1.

6 Using the language

6.1 Program structure and functions

A program is one or more functions and must include `main`. Parameters are C-style, and array parameters are written with `[]`. Recursion is supported.

```
int add(int left, int right) {
    return left + right;
}
```

```
void show(int values[], int count) {
    dor (i : 0 -> count - 1) {
        print(values[i]);
    }
}

int main() {
    print(add(2, 3));
    return 0;
}
```

When you call a function that has an array parameter, pass the array's *name*. Arrays are passed by reference, so the function can change the caller's array. This is how `insertionSort` sorts `data` in place.

6.2 Types, variables and arrays

```
int score = 10;
int values[4] = {8, 3, 5, 1};
values[1] = score - 2;
print(values[1]);
```

- Accepted types: `int`, `float`, `bool`, and `void` (only as a function return type).
- The runtime and the IR generator store every value as an integer. `float` and `bool` are accepted but behave like `int`. A literal such as `3.5` is read as `3`, and `true`/`false` are `1`/`0`.
- Several variables can be declared together: `int a = 1, b, c = 3;` (a variable with no initial value starts at 0).
- Arrays are fixed-size, one-dimensional and zero-indexed. An initializer list `{...}` fills the array from index 0. Indexes are not bounds-checked, so keep every index inside the declared size.
- Comments start with `//` and run to the end of the line.

6.3 Operators and output

Group	**Operators** (lowest to highest precedence)
Logical	`||`, then `&&`
Equality	`== !=`
Comparison	`< > <= >=`
Additive	`+ -`
Multiplicative	`* / %`
Unary	`-x !x`

Zero means false and any non-zero integer means true. Use parentheses to group.

The statement `print(expression);` writes one integer followed by a newline.

6.4 Conditions and loops

```
pecha (score > 50) {
    print(score);
```

```
} warna {
    print(0);
}

dheel (score > 0) {
    score = score - 1;
}

// The final value is included: prints 0 1 2 3 4
dor (i : 0 -> 4) {
    print(i);
}
```

`dor` works in three steps. (1) It evaluates the inclusive end bound once. (2) It evaluates the start value and assigns it to the loop variable. (3) It runs the body while the variable is `<=` the end, adding 1 after each pass. Only increasing ranges are supported. Figure 2 shows how the compiler turns this loop into LLVM basic blocks.

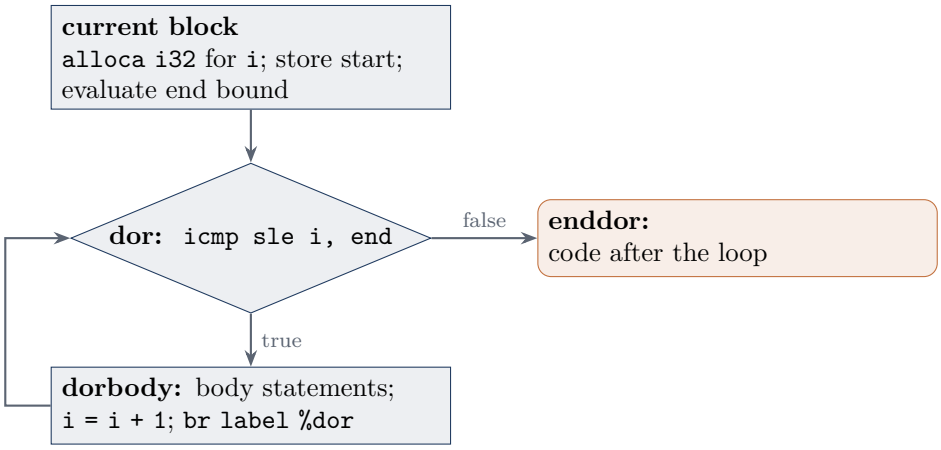


Figure 2: Schematic of how `dor` is lowered to LLVM IR: a header block re-tests `i <= end` before every pass and jumps to `enddor` once it fails. The block names come from the IR emitter in `src/kitelang.y`, which adds a number to each label.

6.5 `palta` – swap

```
palta(values[0], values[1]);
palta(score, values[2]);
```

`palta` swaps the values of two variables or array elements. Both arguments must be assignable, meaning a variable name or an array element. In LLVM IR it becomes two ordinary loads followed by two stores.

7 Examples and testing

7.1 `examples/insertion_sort.kite`

This is the main demonstration. It sorts six integers using `dor`, `pecha` and `palta`, then prints them.

```
// KiteLang insertion sort: expected output is 1 2 4 7 8 9.
void insertionSort(int a[], int n) {
    // Adjacent-swap form of insertion sorting, using KiteLang's dor and palta.
```

```
    dor (i : 0 -> n - 2) {
        dor (j : 0 -> n - 2 - i) {
            pecha (a[j] > a[j + 1]) {
                palta(a[j], a[j + 1]);
            }
        }
    }
}

int main() {
    int data[6] = {9, 4, 7, 2, 8, 1};
    insertionSort(data, 6);
    dor (i : 0 -> 5) {
        print(data[i]);
    }
    return 0;
}
```

7.2 examples/array_smoke.kite

A basic test of arrays and `dor`. It prints the array in its original order.

```
int main() {
    int data[6] = {9, 4, 7, 2, 8, 1};
    dor (i : 0 -> 5) {
        print(data[i]);
    }
    return 0;
}
```

```
./build/kitelang-interpreter examples/array_smoke.kite
./build/kitelang-compiler examples/array_smoke.kite -o build/smoke.ll
clang build/smoke.ll -o build/smoke && ./build/smoke
```

Both should print `9 4 7 2 8 1`, one number per line.

7.3 Writing your own program

1. Create a file, e.g. `myprog.kite`, containing an `int main()` function that ends with `return 0;`.
2. Run it with the interpreter: `./build/kitelang-interpreter myprog.kite`.
3. Compile it natively with `./build/kitelang-compiler myprog.kite -o myprog.ll`. Then build it with `clang myprog.ll -o myprog` and run `./myprog`.

7.4 Automated test

```
make test
```

`make test` does the following:

1. Builds both phases.
2. Runs the interpreter on `insertion_sort.kite`, saves the output to `build/interpreter.out`, and compares it with `tests/expected_insertion_sort.txt` using `diff -u`.

3. Generates `build/insertion.ll` with the compiler.
4. Builds `build/insertion` with Clang and runs it, saving the output to `build/compiler.out`.
5. Compares that output with the same expected file.

Expected file `tests/expected_insertion_sort.txt`

`1`, `2`, `4`, `7`, `8`, `9` – one per line. A zero exit status from `make test` means both phases produced exactly this.

8 Project files

Path	Purpose
`src/kitelang.l`	Flex lexer: keywords, `true`/`false`, identifiers, numbers, operators (`->`, `==`, `!=`, `<=`, `>=`, `&&`, `||`), and `//` comments.
`src/kitelang.y`	Bison grammar and AST construction, plus the Phase 1 tree-walking interpreter, the Phase 2 LLVM IR emitter and the shared `main()` driver.
`src/ast.hpp`	AST node kinds and the `Node` structure shared by the parser and the lexer.
`examples/insertion_sort.kite`	Main Phase 1/Phase 2 sorting demonstration.
`examples/array_smoke.kite`	Basic array and `dor` loop smoke test.
`tests/expected_insertion_sort.txt`	Expected output used by `make test`.
`Makefile`	Build, test and cleanup commands.
`README.md`	Markdown version of this guide.
`README.pdf`	This document.
`docs/README.tex`	LaTeX source of `README.pdf`.
`docs/generate_readme.py`	Earlier ReportLab script that produced a shorter version of `README.pdf` (kept for reference).
`KiteLang_Proposal.pdf`	Original Phase 0 language proposal.
`.gitignore`	Excludes `build/`, `*.ll`, `*.o` and other generated files.
Generated by `make` (not committed):	
`build/parser.tab.cpp`, `build/parser.tab.hpp`	Parser produced by Bison.
`build/lex.yy.cpp`	Lexer produced by Flex.
`build/kitelang-interpreter`	Phase 1 executable.
`build/kitelang-compiler`	Phase 2 executable.
`build/insertion.ll`, `build/insertion`	IR and native program produced by `make test`.
`build/interpreter.out`, `build/compiler.out`	Captured outputs compared by `make test`.

9 Troubleshooting

Problem	Fix
`flex`, `bison` or `clang`: command not found	Install the packages listed in Section 3.
`KiteLang syntax error at line N: ...`	Check braces, semicolons and keyword spelling near line `N`.
`KiteLang error: undefined name: x`	Declare the variable before using it.
`KiteLang error: main function missing`	Add an `int main()` function.
`KiteLang error: wrong argument count`	Call the function with the same number of arguments it declares.
`array argument must be an array name`	Pass an array parameter as a plain name, e.g. `sort(data, 6)`.
Clang rejects `ptr` in the `.ll` file	Your Clang is older than 15; see the Clang version box in Section 3.
Wrong values or crashes with arrays	Use valid zero-based indexes inside the declared size.
Permission denied running a native program	Run `chmod +x build/insertion` and try again.

10 Scope and known limitations

The project targets integer algorithm programs written in one source file. It supports functions, recursion, fixed one-dimensional arrays, conditions, loops and swap. Pointers, dynamic memory, multidimensional arrays, structs/classes, external libraries and multi-file programs are outside the scope.

Phase 2 (LLVM) back-end limits

The interpreter handles the whole language. The IR emitter is narrower. To make sure a program compiles with Clang:

- Write conditions in `pecha`/`dheel` as one comparison, e.g. `a > b` or `x != 0`. The emitter does not lower `&&`, `||`, `!` or unary `-` correctly, so only use them with the interpreter. Write `0 - x` instead of `-x`.
- Declare arrays with a literal size, e.g. `int a[6];`.
- Do not store a comparison result directly in a variable (`int c = a < b;`). Use it only as a condition.

Both example programs stay within these rules.